

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Causes of Poor Generalization

- **Overfitting:** Tree memorizes training data patterns instead of learning general rules.
- **Deep Tree Structure:** Excessive depth leads to highly specific splits that don't generalize.
- **Noisy Data:** Irrelevant or inconsistent features cause misleading splits.
- **Imbalanced Dataset:** If one class dominates, the tree may bias toward it.

Debugging Steps

1. **Check Tree Depth:** Visualize the tree; if too deep, prune or limit depth.
2. **Inspect Features:** Identify redundant or irrelevant features.
3. **Cross-Validation:** Test performance across multiple folds to detect overfitting.
4. **Analyze Errors:** Look at misclassified samples to see if noise or imbalance is the issue.

Optimization Methods

- **Pruning:** Remove branches that add little predictive power.
- **Regularization Parameters:** Limit max_depth, min_samples_split, or min_samples_leaf.
- **Feature Selection:** Use correlation analysis or PCA to reduce irrelevant features.
- **Cross-Validation:** Ensures model stability across different subsets.
- **Ensemble Methods:** Use Random Forests or Gradient Boosting to reduce variance.

Outcome: Improved generalization, balanced accuracy on both training and unseen data.

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Causes of Unstable Results

- **Improper Data Augmentation:** Overly aggressive transformations distort training samples.
- **Vanishing/Exploding Gradients:** Deep networks with poor initialization or unsuitable activations.
- **Batch Normalization Errors:** Incorrect placement or parameter tuning destabilizes training.
- **Learning Rate Issues:** Too high → oscillations; too low → slow convergence.
- **Small Batch Size:** Leads to noisy gradient updates.

Debugging Strategies

1. **Monitor Training Curves:** Plot loss/accuracy per epoch to detect instability.
2. **Check Augmentation:** Ensure transformations (rotation, scaling, flipping) are realistic.
3. **Gradient Inspection:** Track gradient magnitudes to detect vanishing/exploding issues.
4. **Batch Norm Validation:** Verify correct placement (after convolution, before activation).
5. **Experiment with Batch Sizes:** Adjust to stabilize gradient updates.

Optimization Techniques

- **Learning Rate Scheduling:** Use adaptive optimizers (Adam, RMSProp) or decay schedules.
- **Gradient Clipping:** Prevent exploding gradients.
- **Proper Initialization:** Xavier/He initialization for stable convergence.
- **Regularization:** Dropout, L2 weight decay to reduce overfitting.
- **Data Augmentation Tuning:** Apply moderate transformations to improve generalization.
- **Batch Normalization:** Correctly applied, it stabilizes training and accelerates convergence.

Outcome: Stable training, consistent accuracy across epochs, and improved generalization.